



Batforce Billiards System v1.91

A New Hybrid System for Pool Tournament Management, Pairing, and Rating

Leonid Potapov, Falk Hempel
July 27, 2026

Abstract: *The Batforce Billiards System (BBS) is a proof-of-concept hybrid tournament system for local fixed-rack pool events. It combines Swiss-style tournament structure, Elo-style expected-score rating updates, and opponent-adjusted performance interpretation in a browser-based workflow.*

The central BBS-specific element is General Billiards Rating (GBR), a hybrid rating model adapted to fixed-rack matches by combining match outcome and rack-ratio information. This allows the system to distinguish close wins, draws, and dominant rack results while keeping rating updates explainable. BBS also includes Performance Rating (PERF), pairing generation, bye handling, late player changes, correction tools, crash recovery, and CSV export.

BBS is not intended to replace global rating ecosystems such as FargoRate. Its purpose is narrower: to support small and medium local tournaments where transparent calculations, local control, and practical live-event operation matter more than centralized infrastructure.

Keywords: Batforce Billiards System, BBS, General Billiards Rating, GBR, Elo-derived rating, Performance Rating, PERF, Swiss tournament, pool tournament, pairing algorithm, rack differential

Alpha Release: Automatic crash recovery via localStorage. CSV export recommended for archival and cross-device transfer.

1 System Goals

Tournament formats and rating systems solve related but distinct problems. A tournament format determines how players are paired, how many matches they play, and how final standings are produced. A rating system estimates longer-term playing strength from results across matches or events.

Several established concepts already exist. In chess, the Elo rating system uses rating differences to estimate expected scores and then updates ratings from the difference between expected and observed results [1]. Similar rating and match-making concepts have been adapted to online games, table tennis, and other competitive environments [2, 3, 4]. Swiss-system tournaments provide a non-elimination structure in which players remain in the event and are paired against opponents with similar current scores while repeated pairings are avoided whenever possible [5, 6]. In pool, FargoRate provides an established rating ecosystem based on game-level results and a large connected player database [7, 8].

BBS starts from these existing ideas but addresses a different practical problem. Many local pool tournaments need fair pairings, bye handling, rating updates, result exports, and crash

recovery without relying on centralized software, backend infrastructure, or manual spreadsheets.

The contribution of BBS is therefore not the invention of Swiss tournaments or Elo-style rating theory. Instead, BBS introduces a practical hybrid system for local fixed-rack pool tournaments. It combines Swiss-style tournament operation with General Billiards Rating (GBR), a BBS-specific Elo-derived rating model customized for pool by combining match outcome and rack-ratio information.

This document does not claim to provide a globally optimal statistical rating system. It describes a practical local tournament-management system: explainable, fast, able to run without internet access, and calibrated for the operational constraints of small-to-medium billiards events.

1.1 Design Objectives

- **Fair Pairing:** Match players with similar tournament results while avoiding repeat pairings.
- **Skill-Sensitive Rating:** Use GBR to incorporate both match outcomes and rack-level performance.
- **Operational Flexibility:** Handle mid-tournament player additions, removals, and manual corrections.
- **Predictable Timing:** Use fixed rack counts for reliable scheduling.
- **Data Portability:** Use CSV workflows for local persistence and archival.
- **Explainability:** Keep formulas, pairing rules, and rating changes easy to check.

1.2 Design Philosophy

BBS prioritizes practical tournament operation over theoretical optimality. Its core design principles are:

- **Explainability:** Pairings, rankings, and rating changes should be understandable without hidden infrastructure.
- **Venue Independence:** Tournament operation should not depend on external servers, subscriptions, or internet access.
- **Operational Robustness:** The system should tolerate browser refreshes, late player changes, and administrative corrections.
- **Statistical Pragmatism:** The rating model should be useful and stable, even if it is not a full global estimator.

In local pool communities, trust is operationally important. Players are more likely to accept pairings, byes, and rating changes when the rules can be explained directly.

1.3 Rating-System Context

BBS is best understood as an adaptation and integration of established competitive concepts.

Elo-style systems provide the expected-score rating foundation: a rating difference implies an expected result, and ratings change when observed results differ from that expectation [1]. Swiss systems provide the tournament structure: predetermined rounds, score-group pairings, bye handling, and avoidance of repeat opponents where possible [5, 6]. FargoRate demonstrates the value of pool-specific strength estimation at scale, using game-level results across a connected player network [7, 8].

BBS uses these ideas for a different use case: local fixed-rack pool tournaments that need transparent calculations, local control, browser-only operation, and CSV-based archival. The BBS-specific element is General Billiards Rating (GBR). GBR is introduced in this document as a local long-term rating scale derived from Elo-style expected-score logic but extended with

rack-ratio sensitivity for fixed-rack pool matches.

In short, Elo provides the rating foundation, Swiss systems provide the tournament structure, FargoRate provides a pool-specific point of comparison, and BBS introduces a local hybrid implementation for fixed-rack pool tournaments.

Scope of Validity: BBS is primarily intended for local pool tournaments with fixed-rack matches, single-device tournament direction, and CSV-based archival workflow. The current frontend workflow is designed mainly around small and medium local events. The round-planning formulas scale to larger fields, but larger tournaments increase the need for backend storage, multi-device access, and stronger administrative tooling.

2 Tournament Format

2.1 Swiss System Overview

BBS uses a Swiss-style non-elimination format. All players remain in the tournament and are paired each round according to current tournament state. After a predetermined number of rounds, final standings determine the result.

Swiss systems are widely used in chess and other competitive formats. Two core Swiss principles are fixed round count and avoiding repeat pairings whenever possible [5, 6]. BBS adapts these ideas to local pool tournaments.

Compared with single elimination, Swiss avoids early knockout and gives all players multiple matches. Compared with full round-robin, Swiss requires far fewer rounds: a round-robin field of n players requires $n(n-1)/2$ total matches, while Swiss can produce operationally useful standings with a much smaller number of rounds, especially when the goal is a credible top group rather than a complete ordering of the field.

Venue Practicality:

Swiss formats also solve a non-mathematical but very real local-pool problem: players who are not eliminated tend to stay in the building. This keeps tables active, spectators engaged, and the venue busy for the whole event. For many local pool clubs, especially those hosted inside public bars or billiard halls rather than private clubhouses, this matters. A tournament format that keeps players on-site longer can support food, drink, and table revenue for the venue operator, which in turn helps maintain a good relationship between club and establishment.

2.2 Basic Match Terms

A **rack** is one played game of pool. A **match** is the contest between two players in one tournament round.

In BBS, matches use a fixed number of racks. For example, in a 6-rack match, all six racks are played. The match result S_{match} is then determined from the number of racks won R_{won} and the number of racks lost R_{lost} :

$$\begin{array}{lll} \text{win} & S_{\text{match}} = 1 & \text{if } R_{\text{won}} - R_{\text{lost}} > 0, \\ \text{draw} & S_{\text{match}} = 0.5 & \text{if } R_{\text{won}} - R_{\text{lost}} = 0, \\ \text{loss} & S_{\text{match}} = 0 & \text{if } R_{\text{won}} - R_{\text{lost}} < 0. \end{array}$$

Match Points (MP) are the accumulated tournament score from match outcomes. A win gives 1 MP, a draw gives 0.5 MP, and a loss gives 0 MP.

Rack Differential (RD) is the difference between racks won and racks lost:

$$RD = R_{\text{won}} - R_{\text{lost}}. \quad (1)$$

The rack ratio is the proportion of racks won:

$$S_{\text{racks}} = \frac{R_{\text{won}}}{R_{\text{won}} + R_{\text{lost}}}. \quad (2)$$

These terms are used throughout the rating, pairing, and ranking sections.

2.3 Fixed Rack Count Rationale

BBS uses fixed rack counts per match rather than race-to- N formats.

Advantages:

- **Predictable Duration:** Each round completes in bounded time.
- **Reliable Scheduling:** Tournament directors can plan session length more precisely.
- **Consistent Samples:** All matches provide the same number of rack observations.
- **Competitive Incentive:** Players compete for every rack, even when the match outcome is already decided.

Fixed-rack matches were chosen because they are easier to schedule in a local hall than race-to- N formats. In a Swiss tournament, several matches have to finish before the next round can start. A fixed rack count keeps the slowest match from dominating the entire event schedule.

2.4 Draw Outcomes and Result Symmetry

Fixed rack formats enable draw outcomes when players split evenly, for example 3–3 in a 6-rack match. Race-to- N formats force a binary winner even when two players are evenly matched.

Draws are treated as $S_{\text{match}} = 0.5$, making them naturally compatible with expected-score rating models. They also provide a neutral result for close matches, improving result symmetry in short formats.

Fixed rack formats additionally provide rack-ratio information. A 5–1 result and a 4–2 result are both wins, but they contain different information about dominance. BBS uses this difference in the GBR update.

3 General Billiards Rating Model

3.1 GBR as Hybrid Model

BBS introduces General Billiards Rating (GBR), a BBS-specific hybrid rating model derived from the Elo expected-score framework [1]. GBR is not an established external billiards rating standard. It is defined in this document as the local long-term player rating used by BBS.

Like Elo, GBR compares an observed result against an expected score based on the rating difference between two players. Unlike a classical win/loss implementation, GBR is adapted to fixed-rack pool matches by combining two components:

- **Match outcome:** who won, drew, or lost the match.
- **Rack ratio:** how dominant the result was across the fixed number of racks.

This matters because fixed-rack pool matches contain more information than a binary match result. A 5–1 win and a 4–2 win are both wins, but they imply different levels of dominance. GBR uses this rack-level information while preserving the familiar Elo-style interpretation of rating differences, expected scores, and incremental updates.

The term General Billiards Rating is therefore a BBS-defined system term. “General” means that GBR provides one consistent local scale across BBS tournaments, exports, and club

databases. It does not mean that GBR is globally calibrated across independent venues or equivalent to centralized rating ecosystems such as FargoRate.

3.2 Notation and Starting Values

The following notation is used for the GBR model. Player identity is written as a subscript, while rating components are written as superscripts.

Symbol	Meaning
GBR_A^{start}	Player A's GBR before a match
GBR_A^{new}	Player A's GBR after a match
GBR_B^{start}	Player B's GBR before a match
E_A	Expected score for player A
S_{match}	Match outcome: 1, 0.5, or 0
S_{racks}	Proportion of racks won
d	Logistic rating-scale parameter
K_{match}	K-factor for the match-outcome component
K_{rack}	K-factor for the rack-ratio component
$\Delta GBR_A^{\text{match}}$	GBR change for player A from match outcome
$\Delta GBR_A^{\text{racks}}$	GBR change for player A from rack ratio
$\Delta GBR_A^{\text{total}}$	Total GBR change for player A

Each player enters a match with a starting rating GBR^{start} . The match produces a rating change ΔGBR , and the player leaves the match with a new rating GBR^{new} .

Initial GBR values should be assigned from existing local knowledge where possible. The following orientation bands map the BBS rating scale to approximate German pool league levels and regional tournament experience. They are starting-value recommendations, not official league ratings.

Starting GBR	Orientation / German league reference
2200+	National elite / top Bundesliga reference
2100	1. Bundesliga reference
2000	2. Bundesliga reference
1900	Regionalliga / strong regional-tournament winner
1800	Upper Oberliga / strong regional-tournament contender
1700	Oberliga reference
1600	Verbandsliga reference
1500	Landesliga reference
1400	Bezirksliga reference
1300	Kreisliga reference

Regional tournaments typically feature players around 1700–1900 GBR, with winners often around 1900+ GBR. These bands are orientation values for initial seeding and starting-rating assignment only. Individual players may differ substantially depending on specialization, recent form, discipline mix, and local league strength.

3.3 Expected Score

Expected score is calculated using a logistic function with scaling parameter d , default $d = 330$:

$$E_A = \frac{1}{1 + 10^{(GBR_B^{\text{start}} - GBR_A^{\text{start}})/d}} \quad (3)$$

Where E_A is the expected score for player A in the 0–1 range.

3.4 Match Outcome Component

The match-outcome component is:

$$\Delta GBR_A^{\text{match}} = K_{\text{match}} (S_{\text{match}} - E_A). \quad (4)$$

Where K_{match} is the match-outcome K-factor, default 30.

3.5 Rack Ratio Component

The rack-ratio component is:

$$\Delta GBR_A^{\text{racks}} = K_{\text{rack}} (S_{\text{racks}} - E_A). \quad (5)$$

Where K_{rack} is the rack-ratio K-factor, default 20.

Using the same expected score for rack ratio and match outcome is a simplification chosen for explainability. A more complete model could treat rack-level and match-level probabilities separately. For the current BBS version, the simpler model is easier to inspect, easier to explain to players, and sufficient for local tournament use.

3.6 Final Rating Change

The total rating change for player A is:

$$\Delta GBR_A^{\text{total}} = \Delta GBR_A^{\text{match}} + \Delta GBR_A^{\text{racks}}. \quad (6)$$

The new ratings are then:

$$GBR_A^{\text{new}} = GBR_A^{\text{start}} + \Delta GBR_A^{\text{total}} \quad (7)$$

$$GBR_B^{\text{new}} = GBR_B^{\text{start}} - \Delta GBR_A^{\text{total}}. \quad (8)$$

Because Player B's observed match and rack scores are complementary to Player A's scores, Player B receives the negative of Player A's total GBR change. The update is therefore zero-sum within each match: one player's gain is the other player's loss.

Both K-factors are configurable. The default values $K_{\text{match}} = 30$ and $K_{\text{rack}} = 20$ create a 60%/40% weighting: match outcome contributes more, but rack dominance still matters.

4 PERF: Performance Rating

Beyond GBR changes, BBS tracks Performance Rating (PERF) for each player. PERF estimates the rating level implied by a player's rack results against the opponents faced during the tournament.

The concept is related to performance-rating ideas used in chess rating contexts, where achieved score is interpreted relative to opponent ratings and score expectation [1].

4.1 Concept

PERF answers the question: "What GBR level would be expected to achieve this rack result against this opponent?"

For example, if a player rated 1600 GBR defeats an opponent rated 1700 GBR by 5–1, the performance calculation determines what rating would be expected to achieve an 0.833 rack ratio against a 1700-rated opponent.

Match Points (MP) are the primary tournament score: a win gives 1 MP, a draw gives 0.5 MP, and a loss gives 0 MP. PERF is meant to catch cases that MP alone misses. A player who goes 3–1 against strong opponents should not look the same as someone who reaches the same MP total through a much easier path. PERF is therefore useful as a strength-of-schedule and tournament-form indicator.

4.2 Calculation Formula

For each match, PERF is calculated from the rack ratio achieved:

$$PERF_A = GBR_B^{\text{start}} + d \log_{10} \left(\frac{S_{\text{racks}}}{1 - S_{\text{racks}}} \right). \quad (9)$$

Where:

- $PERF_A$ = Performance Rating for player A in this match
- GBR_B^{start} = opponent's starting GBR for the match
- d = scaling parameter
- S_{racks} = proportion of racks won

4.3 Special Cases

The inverse-logistic PERF formula becomes infinite for perfect or zero rack scores. Therefore, BBS applies a finite cap of $3d$:

- Perfect score, $S_{\text{racks}} = 1$, for example 6–0: $PERF_A = GBR_B^{\text{start}} + 3d$
- Zero score, $S_{\text{racks}} = 0$, for example 0–6: $PERF_A = GBR_B^{\text{start}} - 3d$

With the default $d = 330$, the cap is $3d = 990$. A rating difference of 990 implies:

$$\frac{1}{1 + 10^{-990/330}} = \frac{1}{1 + 10^{-3}} \approx 0.999. \quad (10)$$

The cap therefore treats perfect and zero rack results as approximately 99.9% and 0.1% implied performances rather than literal certainty.

The player's tournament PERF is the average of all individual match performances:

$$PERF = \frac{\sum_{i=1}^n PERF_i}{n}. \quad (11)$$

4.4 Relation to Event-Level Performance

A related approach, used in Fargo-style event-performance calculators, is to solve for the single rating that would have been expected to produce the player's total number of games or racks won against the given opponent field.

BBS currently uses a per-match inverse-rating PERF calculation because it is easy to inspect match by match. A future version could replace or supplement this with an event-level solved PERF:

$$\sum_i G_i \frac{1}{1 + 10^{(GBR_i^{\text{opp}} - PERF)/d}} = \sum_i W_i. \quad (12)$$

where G_i is the number of racks played against opponent i , W_i is the number of racks won against opponent i , and GBR_i^{opp} is the opponent's GBR. This would make PERF a direct event-performance estimate while preserving the GBR scale.

4.5 Practical Example

Player A, rated 1600 GBR, plays three matches:

Match	Opponent	Score	PERF
1	1750 GBR	4–2	1849
2	1650 GBR	2–4	1551
3	1800 GBR	5–1	2031

Average PERF: 1810.

Despite starting at 1600 GBR, Player A performed at approximately 1810 level during the tournament.

4.6 Use as Tiebreaker

In Classic mode, PERF breaks ties between players with identical MP. Higher PERF indicates stronger opponents faced, better rack ratios, or both.

PERF incorporates opponent strength, while Rack Differential expresses the visible rack result directly. This makes PERF especially useful when two players have similar match records but faced different paths through the field.

5 Rating Model Interpretation

5.1 Statistical Foundation

The match-outcome component of GBR follows the standard Elo-style interpretation: ratings are adjusted according to the difference between observed score and expected score [1].

The fundamental update equation is:

$$\Delta GBR_A^{\text{match}} \propto (S_{\text{match}} - E_A). \quad (13)$$

The expected score represents the model's prediction, the actual score is the observed outcome, and the difference drives the rating adjustment.

The rack-ratio component extends this idea heuristically by adding margin sensitivity for fixed-rack pool matches. This improves practical differentiation between narrow and dominant wins, but the full hybrid rating model is not a pure global statistical estimator.

GBR should therefore be interpreted as an Elo-derived local rating model introduced by BBS, not as a separately established billiards rating standard. Its statistical foundation comes from expected-score rating logic, while the complete GBR update rule is a BBS-specific hybrid construction. The match-outcome component follows the standard Elo-style idea; the rack-ratio component is a practical extension for fixed-rack pool matches.

5.2 Reading the Update Rule

- **Expected Score:** GBR predicts a result from the rating difference.
- **Observed Result:** The actual match result and rack ratio are compared with expectation.
- **Learning Rate:** K-factors control how strongly ratings react.
- **Equilibrium:** Ratings stabilize when observed results match expected results.

5.3 Local vs Global Optimization

Two fundamentally different approaches exist for computing ratings from match data.

Incremental Updates:

BBS processes matches sequentially, updating ratings after each result. This is computationally cheap, explainable, and suitable for real-time local tournament operation. The trade-off is that ratings depend on update order and do not globally re-optimize historical results.

Global Optimization:

Systems such as FargoRate solve for ratings that best explain a large connected dataset simultaneously. FargoRate describes its ratings as maximum-likelihood ratings that best predict game outcomes across all players in the dataset [7, 8]. BBS uses incremental updates because local pool halls need immediate ratings, inspectable calculations, the ability to run without internet access, and simple CSV workflows.

5.4 Uncertainty

GBR treats ratings as point estimates. Unlike systems such as Glicko, it does not explicitly model rating deviation, volatility, or confidence intervals [9, 10].

This is an intentional trade-off. It keeps the system easier to explain, but it means new players and established players are treated with equal rating confidence unless tournament directors adjust starting GBR manually.

6 Comparison to FargoRate

FargoRate is a widely used pool rating ecosystem. It rates players on a shared scale using game-level results and computes ratings through a global maximum-likelihood approach over a large connected dataset [7, 8].

FargoRate is the appropriate tool if the goal is a cross-venue rating connected to a large shared player database. BBS is not trying to compete with that. The point of BBS is different: a local tournament director can run an event, explain the calculations, export the results, and keep control of the data without needing a central service.

Aspect	Comparison
Data granularity	Both systems use rack-level or game-level pool results.
Update method	BBS uses incremental local updates; FargoRate uses global optimization across a connected dataset.
Infrastructure	BBS runs locally and can be used without internet access; FargoRate is a centralized rating ecosystem.
Transparency	BBS exposes formulas directly; FargoRate uses a closed implementation.
Primary role	BBS serves local tournament management; FargoRate serves cross-venue rating portability.

6.1 Practical Difference

For a venue that already depends on FargoRate, BBS would not replace the established rating scale. It could still be useful as a local tournament tool. For a smaller club without centralized infrastructure, BBS provides a complete local workflow: register players, generate pairings, enter results, update GBR, and export the updated database.

The trade-off is clear. FargoRate benefits from a large shared dataset. BBS benefits from being inspectable and easy to run in the room.

7 Tournament Structure

7.1 Swiss System Implementation

Players face different opponents each round based on current tournament state. After n rounds with p players:

- Total matches: $\lfloor p/2 \rfloor \times n$
- Each player plays n matches, or $n - 1$ if they received a bye
- Repeat pairings are avoided whenever possible

Formal Swiss rules in chess declare the number of rounds beforehand, avoid repeated pairings, and assign a bye if the number of players to be paired is odd [5]. BBS adapts these principles to pool.

7.2 Pairing Order vs Final Standings

BBS separates two questions that are often treated as the same question:

- **Who should play whom next?**
- **How should players be ranked in the final standings?**

These questions do not necessarily require the same ordering rule.

In the ordering notation below, the arrow means “then compare by”. For example, $MP \rightarrow PERF$ means: first sort by Match Points; if players are tied, sort those players by PERF. Likewise, $MP \rightarrow RD \rightarrow PERF$ means: sort by Match Points first, then Rack Differential, then PERF.

Pairing is an operational decision. It determines the next set of matches, not the final placement of the players. In standard BBS mode, pairing uses:

$$MP \rightarrow PERF. \quad (14)$$

Players are therefore ordered first by current tournament score. PERF is used only within the same Match Point group. BBS does not pair purely by estimated playing strength, and it does not pair by raw rack margin.

This matters because pool has significant variance. A weaker player can have the same Match Points as a stronger player after a few rounds, especially in short fixed-rack matches. At the same time, similarly strong players do not necessarily occupy the same tournament position. Pairing must therefore choose whether to emphasize current score, estimated strength, or a mixture of both.

Final standings answer which placement each player earned in the completed event. BBS supports two final-standings modes:

- **Classic:** $MP \rightarrow PERF$. Players are ranked by Match Points first; tied players are ranked by PERF.
- **Rack Differential:** $MP \rightarrow RD \rightarrow PERF$. Players are ranked by Match Points first; tied players are ranked by Rack Differential; remaining ties are ranked by PERF.

Classic mode is more performance-oriented. It uses PERF as the main secondary signal, so players with the same Match Points are ordered by opponent-adjusted performance. This is closer to asking: “Who played the strongest tournament relative to the opponents faced?” The drawback is that PERF is abstract and harder to track during an event.

Rack Differential mode is more player-facing. It still treats Match Points as primary, but uses visible rack margin before PERF. This gives players a clearer short-term goal: win the match, and win as many racks as possible. PERF remains as the next tiebreaker, so opponent strength is not ignored.

The separation is intentional. Pairing can use $MP \rightarrow PERF$ to create stable Swiss-style matchups, while final standings may use Rack Differential when the tournament director wants placements to reflect a more transparent, player-facing competitive result.

In long-term local testing, both final-standings modes often produce similar top groups: stronger players tend to earn more

Match Points, and the table usually scales down naturally. The difference is mainly interpretive and operational: Classic mode leans toward opponent-adjusted performance, while Rack Differential mode leans toward visible competitive achievement.

7.3 Round Requirements and Ranking Reliability

The number of rounds required in a Swiss tournament depends on the goal. Determining a clear winner requires more rounds than identifying a broad top group. For a field of p players, the value $\log_2(p)$ gives a useful lower-bound intuition, but only under a decisive-result model.

In a binary win/loss Swiss tournament, each top-table result tends to split the top score group: one player remains perfect, while the other drops a full point. Under idealized assumptions, approximately $\log_2(p)$ rounds can therefore reduce the field to one undefeated player.

BBS fixed-rack matches allow three match outcomes:

$$S_{\text{match}} \in \{0, 0.5, 1\}. \quad (15)$$

This changes the separation logic. A top-table match can end in a draw, leaving both players close to the top score group. Draws therefore reduce score separation and increase the number of players clustered near qualification boundaries.

The $\log_2(p)$ value should therefore be interpreted as a binary-result lower bound, not as a direct recommendation. Pool outcomes also include rack-level variance, short-match volatility, pairing-path differences, and incomplete comparison structure. Each player faces only a subset of the field.

7.4 Goal-Based Round Model

Different tournament goals require different numbers of Swiss rounds. Determining a credible single winner is harder than identifying a broader qualification group. BBS therefore treats round planning as a goal-dependent problem.

The relevant target may be:

- **#1**: a clear winner or best player
- **Top 3**: the strongest three players
- **Top 25%**: the strongest quarter of the field
- **Top 33%**: the strongest third of the field

These thresholds matter because different tournament stages require different levels of ranking precision. A final winner requires strong score separation at the top. A qualification cut only requires enough confidence that the selected group is stronger than the remaining field.

Let p be the number of players and let $R_0(p)$ be the binary Swiss baseline:

$$R_0(p) = \lceil \log_2(p) \rceil. \quad (16)$$

This baseline reflects the idealized number of rounds needed to separate a field under decisive win/loss outcomes. BBS then adjusts this baseline for the tournament goal, the rack count, and the expected draw rate:

$$R_{\text{goal}}(p, r, \delta) = \max(3, \lceil R_0(p) + B_g + A(r) + D(\delta) \rceil). \quad (17)$$

Where:

- p = number of players
- r = rack count per match
- δ = expected draw rate
- B_g = goal buffer
- $A(r)$ = rack-count adjustment

- $D(\delta)$ = draw-rate adjustment

The goal buffer is:

$$B_g = \begin{cases} +2, & \#1 / \text{clear winner} \\ +1, & \text{Top 3} \\ -1, & \text{Top 25\% of the field} \\ -2, & \text{Top 33\% of the field.} \end{cases} \quad (18)$$

The buffer encodes the practical difference between isolating a single winner and selecting a broader top group. A single winner requires more score separation. A top-quarter or top-third goal tolerates more clustering because the target is a qualification group rather than an exact finishing order.

7.5 Rack Count and Draw Adjustment

The rack-count adjustment reflects the amount of rack-level evidence produced per match:

$$A(r) = \begin{cases} +1, & r \leq 4 \\ 0, & r = 6 \\ -1, & r \geq 8. \end{cases} \quad (19)$$

Shorter matches are more volatile and therefore require more rounds. Longer matches contain more rack-level evidence and can support slightly fewer rounds.

The draw-rate adjustment reflects the fact that draws slow score separation:

$$D(\delta) = \begin{cases} 0, & \delta < 0.10 \\ +1, & 0.10 \leq \delta < 0.25 \\ +2, & \delta \geq 0.25. \end{cases} \quad (20)$$

Here δ is the expected fraction of drawn matches. The values are planning categories, not fitted statistical confidence intervals.

The standard BBS recommendations below assume a moderate draw environment:

$$0.10 \leq \delta < 0.25. \quad (21)$$

This is appropriate for many fixed-rack local tournaments: draws are possible and relevant, but not every close match ends even. If a field is especially draw-prone, add one round. If the field has clear skill separation and draws are rare, the lower end of the recommendation can be used.

7.6 Planning Table: 6-Rack Matches

For 6-rack matches in a moderate draw environment, the model produces the following baseline recommendations. The table shows suggested round counts for different tournament goals. A dash means that the target group is too broad for the field size to require a distinct Swiss-planning recommendation.

Players	#1	Top 3	Top 25%	Top 33%
6–8	6	5	–	–
9–16	7	6	4	3
17–32	8	7	5	4
33–64	9	8	6	5
65–128	10	9	7	6

These values are deliberately conservative for winner determination. With 6 racks, draws and short-match volatility can

leave several players close to the top score group. If the event is casual or the field is clearly mixed-skill, one fewer round may be acceptable.

7.7 Planning Table: 8-Rack Matches

For 8-rack matches in a moderate draw environment, the increased rack sample provides more information for Rack Differential and PERF. The table shows suggested round counts for different tournament goals. A dash means that the target group is too broad for the field size to require a distinct Swiss-planning recommendation.

Players	#1	Top 3	Top 25%	Top 33%
6–8	5	4	–	–
9–16	6	5	3	3
17–32	7	6	4	3
33–64	8	7	5	4
65–128	9	8	6	5

For 8-rack events, the table can be read as the lower, more time-efficient recommendation. If many matches are expected to end 4–4, add one round, especially when the goal is to determine #1 or Top 3.

7.8 Interpreting the Recommendations

The planning tables are not formal confidence guarantees. They are operational recommendations based on four effects:

1. binary Swiss depth, represented by $\lceil \log_2(p) \rceil$
2. goal strictness, represented by B_g
3. rack-level evidence, represented by $A(r)$
4. draw-related score clustering, represented by $D(\delta)$

A tournament director should choose the goal column before the event:

- Use **#1** if the event should produce a credible single winner.
- Use **Top 3** if the event mainly determines podium or prize positions.
- Use **Top 25%** if the Swiss stage is used for qualification or seeding.
- Use **Top 33%** if the goal is a broad top group rather than precise ordering.

The recommendation should be adjusted upward when:

- the field is balanced
- many draws are expected
- the rack count is short
- the winner matters more than the top group
- prize or qualification boundaries are narrow

It can be adjusted downward when:

- the field has clear skill separation
- the event is casual
- the goal is broad seeding rather than exact ranking
- time constraints are more important than ranking precision

7.9 Total Racks and Variance

Total racks per match directly affect result reliability. Shorter matches are more volatile: a weaker player needs fewer favorable racks to draw or win. Longer matches reduce variance but increase round duration.

For even fixed-rack matches, draws are possible. It is useful to distinguish:

- **Outright upset:** the weaker player wins more racks
- **Non-win for stronger player:** the weaker player wins or draws

The second measure is more relevant for Swiss standings because a draw still prevents the stronger player from earning a full match point.

Draws matter because they reduce score separation. In a decisive-result Swiss tournament, a top-board match between two perfect-score players leaves only one player on a perfect score. In a fixed-rack format, the same match can end in a draw, leaving both players close to the top score group. This increases the number of players clustered near qualification boundaries and makes tiebreakers more important.

The key operational conclusion is robust: fewer racks and higher draw rates require more rounds or stronger tiebreakers to produce credible standings.

7.10 Prestige Score

Prestige Score is an optional tournament-series or participation score. It rewards:

- rounds played
- match points earned
- positive GBR gains

Prestige Score is shown in final tables and CSV exports when enabled. It does not affect pairing, final standings order, or GBR calculations.

7.11 Event Score

Event Score is an optional final-table reporting metric on a 0–100 style scale. It is disabled by default and shown only in the final table and CSV export when enabled.

Event Score is descriptive only. It is not the player's final placement, not a rating system, and not a tiebreaker.

$$\text{Event Score} = \text{round}(100FQ) \quad (22)$$

$$Q = 0.70 \frac{MP}{R} + 0.20 RD_{\text{norm}} + 0.10 PERF_{\text{norm}} \quad (23)$$

Where:

- MP = Match Points
- R = total tournament rounds
- RD_{norm} = normalized Rack Differential
- $PERF_{\text{norm}}$ = normalized PERF value
- F = field-strength factor

RD_{norm} is the player's Rack Differential divided by the maximum possible absolute Rack Differential for the event, clipped to the 0–1 range. $PERF_{\text{norm}}$ is a scaled comparison of the player's PERF against the tournament field average. The field-strength factor F adjusts the descriptive Event Score relative to the average strength of the registered club database. If no meaningful club database is available, F should be treated as 1.

Event Score does not affect pairing, standings order, tiebreaking, or GBR changes.

8 Pairing Algorithm

8.1 Conceptual Pairing Problem

Swiss pairing can be understood as a constrained optimization problem. In each round, the system must divide the active

player field into legal pairings while balancing several competing objectives:

1. pair players with similar tournament results
2. avoid repeat opponents
3. respect bye eligibility
4. handle newly added players
5. preserve meaningful ranking progression
6. produce pairings quickly enough for live tournament use

The ideal version of the algorithm would evaluate complete round pairings globally. That means it would not choose opponents one player at a time. Instead, it would search over possible pairing sets and choose the complete round with the lowest total cost.

8.2 Global Round Optimization

Let P be a complete set of pairings for one round. The conceptual optimization problem is:

$$P^* = \arg \min_{P \in \mathcal{P}} C(P) \quad (24)$$

where $\mathcal{P}$ is the set of legal complete pairings. A legal complete pairing uses each active player at most once, respects bye eligibility, and contains no illegal repeated byes.

The total pairing cost is:

$$C(P) = \sum_{(i,j) \in P} c(i,j) + C_{\text{bye}}. \quad (25)$$

The pair cost $c(i,j)$ measures how undesirable it is to pair players i and j :

$$c(i,j) = w_{\text{repeat}} R_{ij} + w_{\text{mp}} \Delta MP_{ij} + w_{\text{perf}} \Delta PERF_{ij}. \quad (26)$$

The repeat-opponent indicator R_{ij} is defined as:

$$R_{ij} = \begin{cases} 1, & \text{if players } i \text{ and } j \text{ have already played each other,} \\ 0, & \text{otherwise.} \end{cases} \quad (27)$$

Where:

- R_{ij} is the repeat-opponent indicator
- $\Delta MP_{ij} = |MP_i - MP_j|$
- $\Delta PERF_{ij} = |PERF_i - PERF_j|$
- the w -terms are weights controlling priority

A repeated opponent is therefore not treated as an impossible pairing in the conceptual cost model. It receives a large penalty through $w_{\text{repeat}} R_{ij}$, so the algorithm avoids repeats whenever a reasonable non-repeat option exists. If the player pool or round count makes repeats unavoidable, a repeat pairing may still be selected rather than failing to generate a round.

In this formulation, the algorithm is not merely trying to find a legal opponent for each player. It is trying to find the best complete round under Swiss constraints.

8.3 Backtracking and Exact Search

A direct implementation can use recursive backtracking:

1. choose the highest-priority unpaired player
2. try each legal opponent
3. recursively complete the rest of the round
4. reject branches that violate hard constraints
5. keep the complete pairing set with minimum total cost

This approach is conceptually clean and can find a lower-cost complete pairing set than a simple greedy pass. The drawback is combinatorial growth. The number of possible complete pairings increases rapidly with player count, especially when the system allows soft constraints instead of rejecting every imperfect pairing.

For small fields, exact backtracking is realistic. For larger local tournaments, it may require pruning, branch-and-bound, or conversion to a weighted matching problem.

8.4 Current Greedy Implementation

The current frontend implementation uses a cost-based greedy approximation. It processes players in pairing order, evaluates available opponents, and selects the lowest-cost candidate for each player.

This is a technical simplification for the local app. It does not change the underlying design idea: the pairing problem is still treated as cost minimization under Swiss constraints. The greedy algorithm is a fast approximation of that optimization problem.

The limitation is that an early locally optimal choice can remove an opponent needed for a better complete round solution later.

8.5 Development Notes

Early versions used recursive backtracking for pairings. This produced strong pairings, but it was unnecessarily complex for the intended tournament sizes and could become slow as the number of players increased.

The current cost-based greedy approach is easier to inspect and fast enough for live use. It is not theoretically perfect, but in the expected local range it is predictable enough for practical tournament direction.

One practical lesson from testing was that pairing and final standings should not necessarily use the same ordering rule. PERF is useful for pairing because it incorporates opponent strength and helps estimate fair next-round matchups. Rack Differential is useful for final standings because it is tangible, player-facing, and easy to verify during the event. This led to the v1.91 separation between pairing order and final standings.

8.6 Round 1 Seeding Modes

Round 1 can be generated using one of several seeding methods:

- **Random:** Players are shuffled and paired sequentially.
- **Cross GBR:** The top half of the field by GBR is paired against the bottom half.
- **GBR Balanced:** Players are sorted by GBR and processed through the cost-based pairing logic to create rating-balanced initial matchups.
- **Manual Selection:** Players are paired in manually entered order.

From Round 2 onward, normal Swiss system pairing is used regardless of Round 1 method.

8.7 Three-Phase Pairing Procedure

Round generation follows three phases.

8.7.1 Phase 1: Bye Assignment

If player count is odd, one player receives a bye. The bye is assigned to the lowest-ranked eligible player in the current standings. Players who have already received a bye and players freshly joining in the current round are not eligible.

Example: In a 13-player tournament, BBS assigns one bye per round. A player who has just been added late is protected

from receiving the immediate bye, so the bye goes to the lowest-ranked eligible existing player who has not yet received a bye.

Bye assignment is deterministic: the same tournament state always produces the same bye.

8.7.2 Phase 2: New Player Pairing

Players added after tournament start are paired before the standard pairing pass. This prevents newly added players from receiving an immediate bye and gives them an opponent of roughly similar strength.

The rating signal is average PERF when available, otherwise current GBR.

8.7.3 Phase 3: Cost-Based Greedy Pairing

Remaining players are processed in pairing order. For each unpaired player, the system evaluates available opponents and selects the candidate with the lowest pairing cost.

8.8 Standard Pairing Order

Standard v1.91 pairing order is:

$$MP \rightarrow PERF \rightarrow ID \quad (28)$$

This pairing order is used regardless of whether the displayed standings use Classic mode or Rack Differential mode.

Rack Differential is deliberately excluded from standard pairing order. It remains available as a final-standings tiebreaker, but it does not influence next-round pairing in default v1.91 behavior.

8.9 Standard Pairing Cost

The current greedy implementation uses the following pair cost:

$$C = P_{\text{rep}} + 10000 \Delta MP + \Delta PERF. \quad (29)$$

Where:

- P_{rep} is the repeat-opponent penalty
- ΔMP is the Match Point gap
- $\Delta PERF$ is the PERF gap

Where $P_{\text{rep}} = 100000$ if the players have already played each other, otherwise 0. The hierarchy reflects Swiss system logic: match record matters most, repeat opponents are avoided, and PERF resolves ambiguous cases between players with similar records.

8.10 Legacy Rack-Differential Pairing

Version 1.91 preserves a legacy rack-differential pairing mode for historical reproducibility.

When enabled, pairing order becomes:

$$MP \rightarrow RD \rightarrow PERF \quad (30)$$

and rack-differential gaps are added to the pairing cost.

This mode is off by default. It is intended for reproducing older pairing behavior, not as the standard pairing recommendation. This does not affect the separate use of Rack Differential as a final-standings tiebreaker.

8.11 Repeat Handling

If repeat pairings become unavoidable, the algorithm allows repeat opponents with a penalty rather than failing to generate a round.

Example: In a small 8-player event, repeat pairings are usually avoidable for a short tournament. In a 10-round event with only

8 players, however, repeats eventually become unavoidable. BBS therefore penalizes repeat pairings rather than refusing to generate a round.

This repeat handling applies to opponent pairings only. Repeated byes remain illegal.

Tournament directors can use the Manual Pairing Editor when the automatic solution is legal but undesirable.

8.12 Performance Characteristics

The current implementation runs in $O(N^2)$ per round, which remains well within interactive use for the intended local tournament sizes. This complexity refers to the current greedy implementation, not to the conceptual exact optimization problem.

The conceptual pairing problem is a global optimization problem. The current greedy algorithm is a practical approximation: it produces usable pairings quickly, while leaving room for future exact backtracking or weighted-matching implementations.

9 Bye Assignment

9.1 Purpose and Mechanics

Swiss tournaments with odd player counts require one player per round to receive a bye. In formal Swiss systems, an odd number of players results in one unpaired participant receiving a pairing-allocated bye [5].

In BBS, a bye functions as an automatic match win without an opponent.

9.2 Assignment Rules

The bye is assigned to the lowest-ranked eligible player in the current standings. The rule is deterministic given tournament state: there is no random bye allocation and no tournament-director discretion required during normal operation.

A player is eligible for a bye only if:

1. they are active in the current round,
2. they have not received a bye in any previous round,
3. they did not freshly join the tournament in the current round.

Among eligible players, BBS assigns the bye to the lowest-ranked player. This minimizes competitive impact while keeping the rule simple and reproducible.

A repeated bye is not allowed. Once a player has received a bye, they are excluded from all future bye assignment.

If every active player has already received a bye, the tournament has exceeded the no-repeat-bye capacity of the current field. In that case, the tournament director must reduce the remaining round count, add eligible players, or resolve the situation manually before continuing.

9.3 Bye Effects

A player receiving a bye:

- receives 1 Match Point
- receives no racks won / lost for RD purposes.
- receives no GBR change
- receives no PERF contribution

A bye awards 1 Match Point but does not contribute to Rack Differential, GBR, or PERF. It is treated as a tournament-score assignment, not as a played rack result.

10 Operational Features

10.1 Player Addition

Players can join after tournament start. Added players enter a pending queue and become active when the next round is generated.

A newly added player:

- receives starting GBR from the club database or manual entry
- is protected from receiving a bye in their first active round
- has previous rounds marked as not played
- accumulates GBR and tournament statistics only from their first active round onward

10.2 Player Deletion

Players can be removed from an active tournament to correct accidental imports, registration errors, or administrative mistakes.

When a player is deleted, the system removes their matches, opponent references, and tournament statistics. If the current round is affected, pairings are regenerated for the remaining players.

This preserves tournament integrity while allowing practical correction of live-event mistakes.

10.3 Manual Pairing Editor

The Manual Pairing Editor is an emergency override tool for tournament directors.

It allows direct manipulation of pairings while still showing warnings for:

- repeat matchups
- duplicate players
- illegal or questionable byes
- other pairing conflicts

The editor is useful for small-tournament edge cases, table constraints, player requests, or rare cases where the automatic greedy pairing is legal but undesirable.

10.4 Round 1 Seeding

Before tournament start, the tournament director can choose the Round 1 seeding method (in accordance to 8.6):

- **Random:** Neutral and simple.
- **Cross GBR:** Strong players open against weaker players.
- **GBR:** Balanced opening pairings by rating.
- **Manual Selection:** Full tournament-director control.

After Round 1, standard Swiss pairing takes over.

10.5 Undo Advance

Version 1.91 includes an emergency Undo Advance feature.

When advancing to the next round, the system stores a snapshot of the previous state. If the tournament director notices an immediate mistake, the tournament can be restored to the state before advancement.

Undo Advance is not a full undo history. It is a single-step safety mechanism for live tournament direction.

10.6 Emergency Autosave Clear

An emergency autosave clear option removes local crash-recovery data. This is useful when recovery data is stale, corrupted, or no longer wanted.

10.7 Table Management

Matches can be assigned table numbers or custom table labels. This supports venues with numeric, alphabetic, or custom table naming.

Table labels are shown in match cards to help players find their assigned table quickly.

11 Data Management

11.1 Import/Export Architecture

As a frontend-only application, persistence relies on CSV import/export and browser-local autosave.

Club Database Import:

The club database uses a simple Name/GBR CSV format. It provides the player pool for tournament registration and supplies the club average GBR used by the optional Event Score field-strength factor.

Results Export:

The results export produces a CSV containing tournament metadata, round-by-round results, final standings, GBR changes, and optional reporting fields such as Prestige Score and Event Score.

Round columns use compact notation:

- 1: win
- 0.5: draw
- 0: loss
- —: not played or joined late

GBR Database Export:

The GBR database export merges tournament results with the original club database. Participants receive updated GBR values, non-participants are preserved, manually added players are included, and the result is exported as a ready-to-import Name/GBR CSV.

Image Export:

The application also supports image export for sharing standings or final results. The layout adapts to tournament size to maintain readability.

11.2 Workflow

A typical local workflow is:

1. Import club database CSV
2. Configure tournament settings
3. Register players
4. Select Round 1 seeding method
5. Run tournament
6. Export results
7. Export updated GBR database
8. Use updated database for the next tournament

11.3 Autosave and Persistence

localStorage provides automatic crash and reload recovery within the same browser context.

Version 1.91 stores the full tournament state, including players, rounds, match results, configuration, pending players, the Undo Advance snapshot, and the imported club database.

Including the club database in autosave is important because Event Score depends on the club average GBR.

11.4 Crash Recovery

On app load, the system checks for a valid autosave snapshot and restores it if possible.

Active tournaments return to the round view. Configured but not yet started tournaments return to setup. Invalid or empty snapshots are ignored.

CSV export remains recommended for:

- permanent archival

- backup redundancy
- cross-device transfer
- recovery after browser data deletion
- long-term tournament records

12 Known Limitations

12.1 Rating Model Simplifications

No Rating Uncertainty: GBR treats ratings as point estimates. It does not model rating deviation, volatility, or confidence intervals as systems such as Glicko do [9, 10]. New and established players therefore carry the same rating confidence unless starting values are adjusted manually.

Local Player Pool Dependency: Rating quality depends on local player pool size, diversity, and tournament frequency. Small isolated pools may develop compression, inflation, or unstable calibration. Cross-venue calibration requires manual anchoring or shared backend infrastructure.

Rack Expectation Approximation: PERF interprets rack-level outcomes through the same logistic structure used for expected score. This is explainable, but it remains an approximation and may deviate under specific break advantages, playing styles, or table conditions.

12.2 Pairing Algorithm Limitations

The current greedy pairing algorithm does not guarantee a globally optimal Swiss pairing. It trades optimality for speed, explainability, and reliability during live events.

Typical edge cases include small fields, many-round events, exhausted opponent pools, unavoidable repeat opponents, or cases requiring the Manual Pairing Editor.

Standard pairings use $MP \rightarrow PERF$, while displayed standings may use Rack Differential. A player may therefore appear higher in the visible table while still being paired by MP and PERF.

12.3 Structural Constraints

Persistence is browser-local. localStorage protects against refreshes and crashes, but data remains device-specific and vulnerable to storage deletion. CSV export remains necessary for archival and transfer.

The frontend-only version cannot synchronize devices, support multiple live editors, or maintain centralized tournament history.

12.4 Event Score Limitations

Event Score is a descriptive reporting metric, not a rating system, placement rule, or tiebreaker. It depends on tournament configuration, rounds, rack count, field size, and club average GBR. It should not be interpreted as a replacement for GBR.

12.5 Acceptable Trade-offs

These limitations are acceptable for the current proof of concept. BBS is meant to be understandable, usable during a live event, and recoverable through CSV files. It is not yet a hosted tournament platform.

13 Software Architecture

13.1 Architecture

BBS is a self-contained frontend system built with React. It uses local browser state for tournament operation, localStorage for crash recovery, and browser file APIs for CSV and image export.

No backend is required for the current alpha version.

13.2 Configuration

Core configuration includes the GBR scale parameter d , K-factors, rack and round count, ranking system, Prestige Score settings, Event Score display, Round 1 seeding method, and the legacy Rack-Diff Pairing option.

The default v1.91 configuration keeps Legacy Rack-Diff Pairing disabled.

13.3 State Model

The application tracks players, configuration, rounds, match results, pending players, current round, imported club database, and recovery snapshots. Derived statistics are recalculated from match history when results change.

13.4 Ranking and Pairing Separation

Version 1.91 separates final standings comparison from pairing comparison.

Final standings:

- Classic: $MP \rightarrow PERF$
- Rack Differential: $MP \rightarrow RD \rightarrow PERF$

Pairing:

- Standard: $MP \rightarrow PERF$
- Legacy option: $MP \rightarrow RD \rightarrow PERF$

This allows Rack Differential to remain a transparent standings tiebreaker without distorting standard Swiss pairings.

13.5 Crash Protection and Compatibility

Autosave protects against refreshes, tab closures, and browser crashes by storing a local snapshot and restoring it on startup when valid. Version 1.91 also includes the club database and Undo Advance snapshot in autosave.

BBS targets modern desktop browsers with ES6+, localStorage, Canvas API, Blob API, and FileReader API support. A desktop or tablet-sized display is recommended.

14 Use Cases and Limitations

14.1 Ideal Applications

BBS is best suited for small and medium local events: weekly club tournaments, single-day Swiss events, or multi-day events managed from one browser.

It is especially useful when the tournament director wants local control, CSV import/export, fixed-rack matches, and a locally maintained rating database.

The frontend workflow is designed mainly for roughly 6–30 players. The round-planning formulas scale to larger fields, but larger tournaments increase the need for backend storage, multi-device access, and stronger administration tools.

14.2 Not Recommended

BBS is not recommended for remote or distributed events requiring real-time synchronization, or for tournaments where several directors must edit the same event from different devices.

Mission-critical tournaments should not rely on browser state alone. CSV export should be treated as the permanent record until backend storage exists.

14.3 Alpha Constraints

The current version can recover from browser refreshes and crashes within the same browser context. It does not provide cross-device continuity, cloud backup, centralized tournament history, or collaborative live editing.

Backend development is necessary for production multi-user scenarios, persistent player databases, and tournament archives.

14.4 Local Deployment Philosophy

The current design can run in the room without subscription, internet access, or third-party services. Tournament directors keep control over player data, rating history, and archival exports.

That control matters in small communities. BBS is not only a rating formula; it is a practical local tournament workflow.

15 Future Development

The first backend priority is persistent player records and tournament archives. CSV export works, but it puts archival responsibility on the tournament director. Backend storage would reduce the risk of losing history and make GBR continuity less dependent on manual file handling.

Core production infrastructure would include a backend database, user authentication, persistent GBR storage, tournament archives, and a hosted API.

After persistent storage, live features become useful: real-time standings, multiple tournament directors, mobile access, player-facing result submission, and spectator views.

Future formats could include Swiss plus knockout brackets, tournament series, single-match GBR recording, tiered ranking tournaments, and season-long Prestige Score leaderboards. Swiss plus knockout is the most natural next format: Swiss rounds produce seeding, then top players finish with a bracket. No timeline is committed. Development should follow actual tournament use, not feature completeness alone.

16 Conclusion

BBS v1.91 is a working proof of concept for local pool tournaments that need fair pairings, understandable ratings, and reliable operation without a backend.

Its main value is integration: fixed-rack Swiss structure, GBR rating updates, PERF tiebreaking, deterministic byes, crash recovery, manual correction tools, and CSV export in one browser-based workflow.

GBR provides a BBS-specific Elo-derived rating update for fixed-rack pool matches, while PERF adds an event-level performance signal relative to opponent strength. Version 1.91 also separates standard pairing from final standings, keeping Rack Differential available as a transparent standings tiebreaker without letting early rack margins distort Swiss pairings.

The next step is persistent storage and multi-device support while preserving the explainability that makes the current version usable.

References

- [1] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. New York: Arco Publishing, 1978.
- [2] Riot Games. *MMR, Rank, and LP*. 2026. URL: <https://support-leagueoflegends.riotgames.com/hc/en-us/articles/4405781372051-MMR-Rank-and-LP> (visited on 05/02/2026).
- [3] International Table Tennis Federation. *ITTF Table Tennis World Ranking Regulations*. 2024. URL: <https://wtwebcmsprod.blob.core.windows.net/technicaldocuments/ITTF%20Table%20Tennis%20World%20Ranking%20Regulations.pdf> (visited on 05/02/2026).
- [4] USA Table Tennis. *USATT Ratings*. URL: <https://www.usatt.org/usatt-ratings> (visited on 05/02/2026).
- [5] FIDE. *C.04.1 Basic Rules for Swiss Systems*. URL: <https://handbook.fide.com/chapter/C0401202507> (visited on 05/01/2026).
- [6] FIDE. *C.04.2 General Handling Rules for Swiss Tournaments*. URL: <https://handbook.fide.com/chapter/GeneralHandlingRulesForSwissTournaments202602> (visited on 05/01/2026).
- [7] FargoRate. *FargoRate*. URL: <https://fargorate.com/> (visited on 05/01/2026).
- [8] Mike Page. *Fargo Ratings: A Look Under the Hood*. 2024. URL: https://drdavepoolinfo.com/resource_files/Mike_Page_FargoRate_description.pdf (visited on 05/01/2026).
- [9] Mark E. Glickman. *The Glicko System*. 1995. URL: <https://www.glicko.net/glicko/glicko.pdf> (visited on 05/01/2026).
- [10] Mark E. Glickman. "Parameter Estimation in Large Dynamic Paired Comparison Experiments". In: *Journal of the Royal Statistical Society: Series C* 48.3 (1999), pp. 377–394.